

Production-Quality Interactive Course Labs, Built with AI

How AI coding agents let one instructor ship a fully interactive, zero-install course platform — *SYSEN 5470: Network Science & Applications for Systems Engineering*



Tim Fraser, Ph.D.
Systems Engineering · Cornell University
tmf77@cornell.edu · timothyfraser.com/netsci

11 interactive labs, live on the course site

21 planted-story project datasets

2 languages taught in lockstep (R + Python)

17 packages preloaded in the browser

\$0 servers, accounts, or hosting

THE PROBLEM

Students learn networks by clicking, breaking, and rebuilding them — not by reading PDFs. But one production-quality interactive lab takes weeks of web engineering: simulation code, instant-feedback questions, mobile support. A full course needs dozens — plus datasets, codebooks, and teaching code in two languages. That is months of engineering no teaching schedule allows.

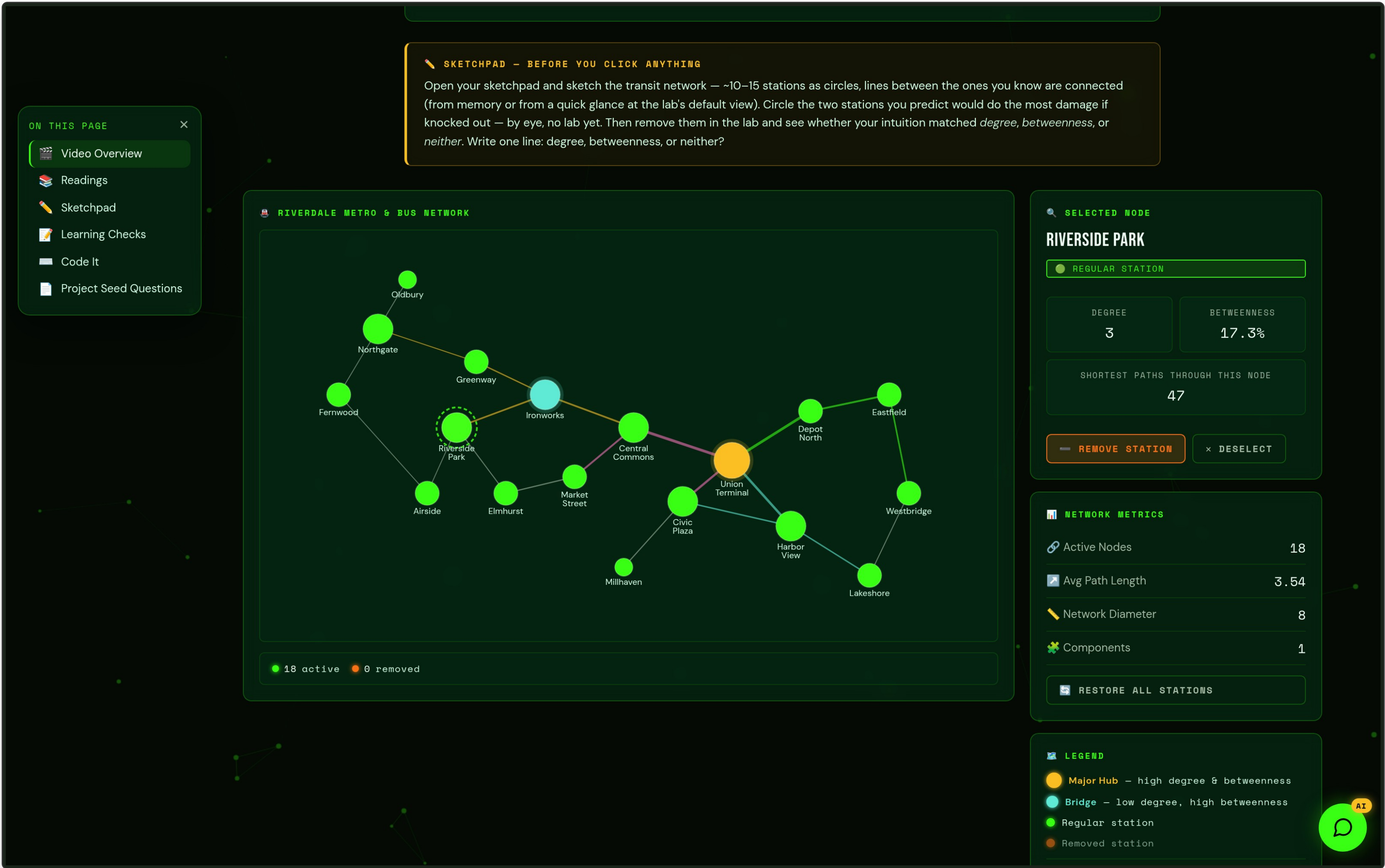
THE AI WORKFLOW

The instructor directs and reviews; AI agents do the engineering.

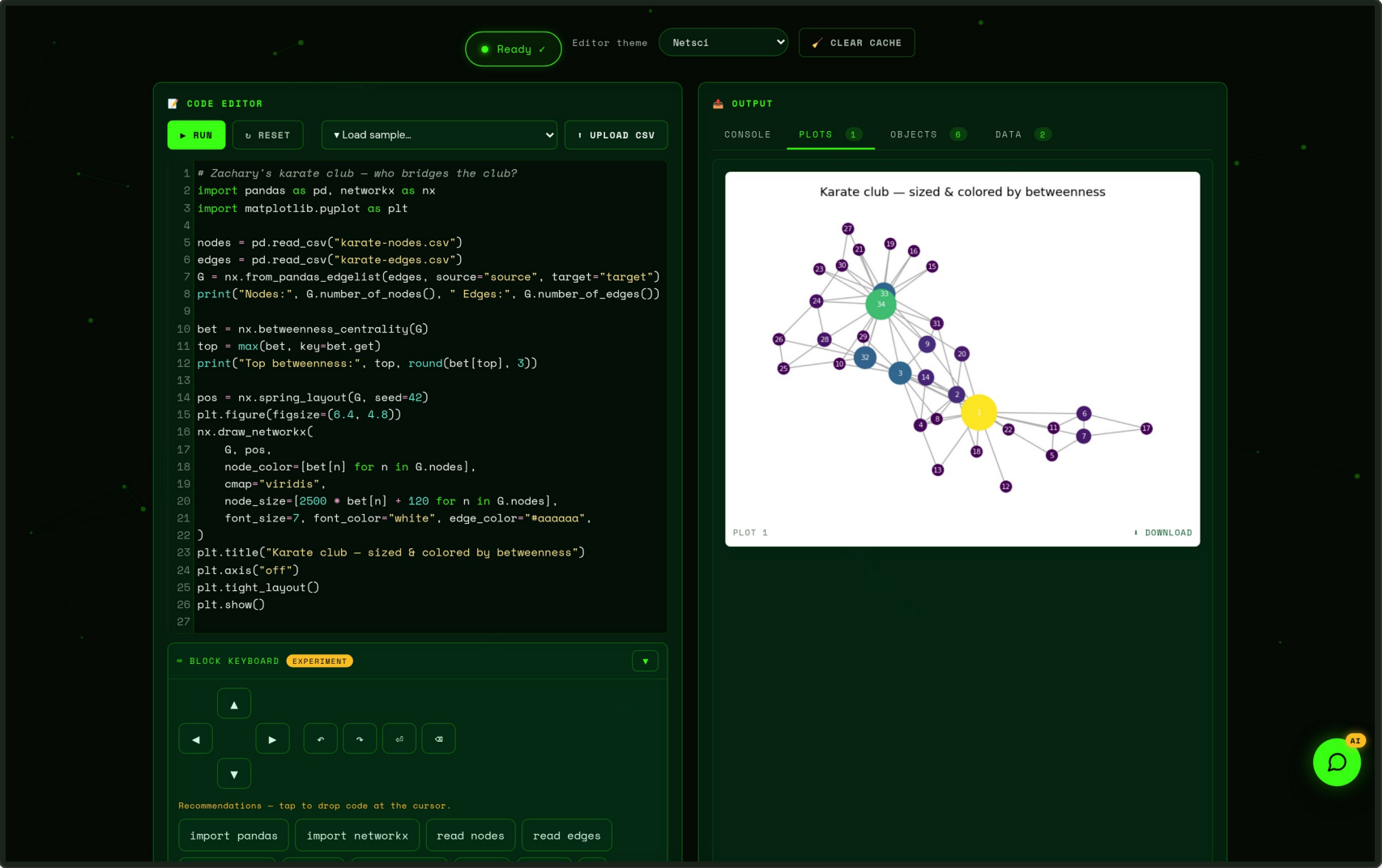
- 1 Encode conventions once** — Reusable agent "skills" capture teaching style, dataset standards, and the course glossary.
- 2 AI drafts each artifact** — A lab page, a dataset generator, or a matched R + Python teaching script.
- 3 Verify automatically** — Headless-browser tests; answer keys recomputed from the rendered page; R and Python outputs cross-checked to match.
- 4 Instructor reviews** — A pedagogy, tone, and accuracy pass on every artifact before it ships.
- 5 Ship free & static** — GitHub Pages hosting; CI rebuilds the student repo and AI study companion automatically.

Built with agentic AI coding tools (Claude Code). Every artifact is verified; none is hand-coded.

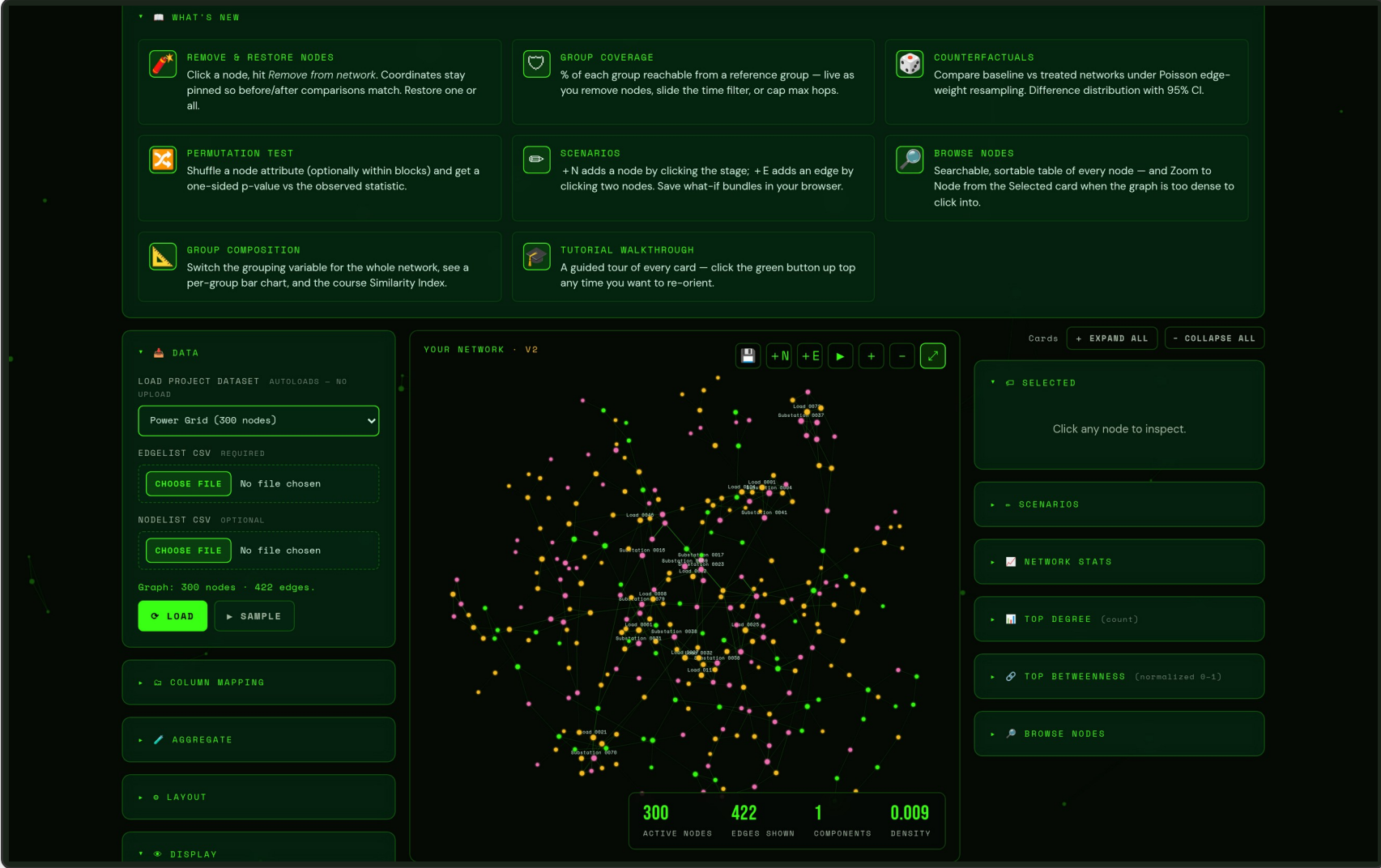
THE LABS, LIVE — EVERY ONE A MANIPULABLE ARTIFACT, NOT A PDF



Case-study lab “Why does betweenness matter?” — remove a transit station and watch path lengths, components, and criticality shift live. Learning checks are recomputed from the rendered network, so answers can never drift.



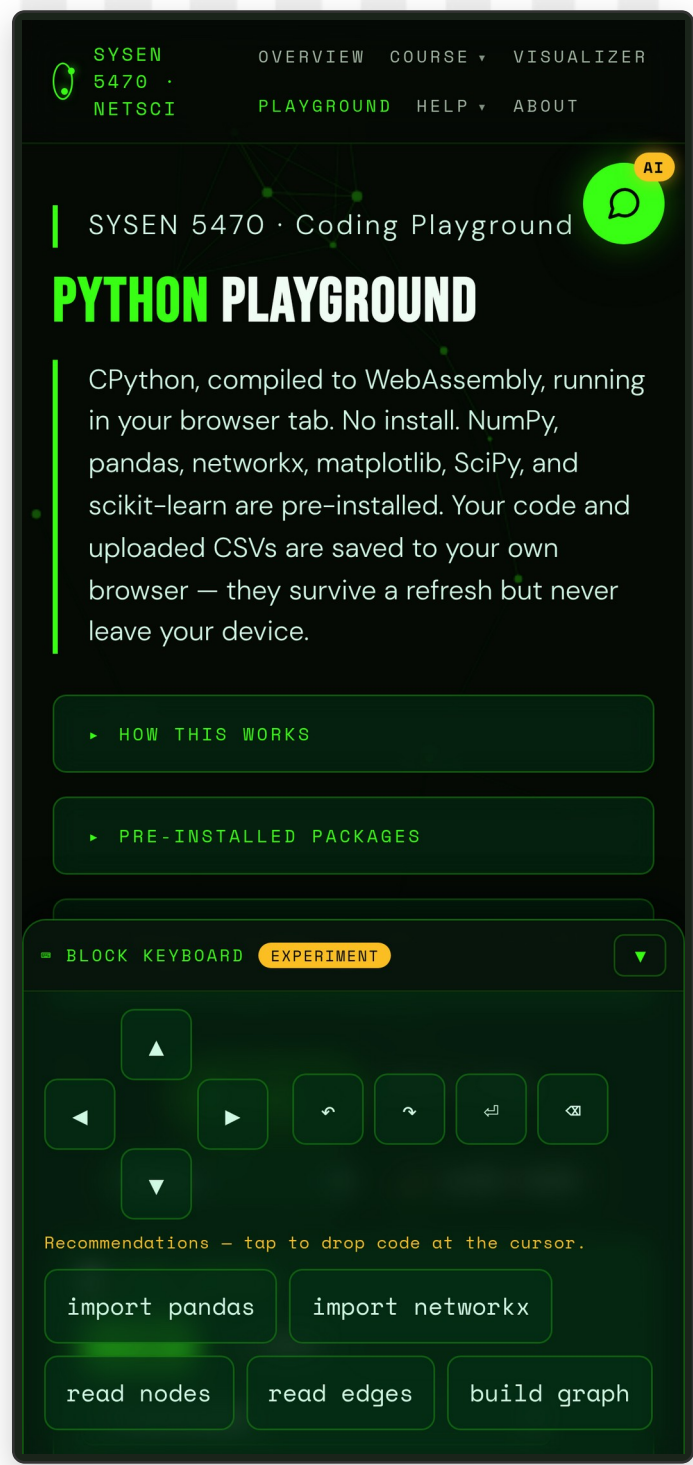
In-browser R & Python playgrounds (WebAssembly): real code, real plots, on the student’s own device — no install, no login.



Network Visualizer: point-and-click failure cascades and counterfactuals — then export the runnable script that reproduces every click.

WHAT STUDENTS GET

- Real R & Python in the browser — zero install, no accounts, works on any device.
- Labs that push back: remove a node, watch the metrics shift, get instant feedback.
- Explore visually, then export a complete runnable script and keep going in code.
- A Socratic AI study companion that asks questions back instead of giving answers.



Code on a phone.
A tap-to-code block keyboard — D-pad cursor, undo/redo, and context-aware code chips — lets students write real code from the bus, the gym, or the back row.

TRY IT RIGHT NOW



Course site



Network Visualizer

21 synthetic project datasets, generated by AI under a strict codebook standard — each hides a “planted story” that survives naive analysis, and ships with R + Python loaders.

